PS 01: Self-Healing Tool Registries Under Runtime API Schema Drift

Problem Background

Microservices and third-party APIs change continuously, but production agents rely on static OpenAPI or Swagger specifications translated into function-calling schemas. When downstream teams push breaking changes, such as renaming a parameter, nesting a response, adding a mandatory OAuth header or changing a field's type, the agent crashes with 4xx errors, loops on the same bad payload, or passes hallucinated defaults that corrupt data. Post-mortems show that interface drift accounts for over 40% of runtime tool-call outages in long-lived agents.

Problem Statement

Build an autonomous, self-healing tool interface. When an API returns an error such as 422 Unprocessable Entity or 400 Bad Request, the system should catch the breaking change at runtime, deduce the new schema, hot-patch the agent's tool definitions in memory, and complete the workflow without downtime. Standard frameworks either throw an unhandled exception or return raw JSON errors to the agent, which then retries the same invalid parameters until it runs out of iterations.

Objectives

- Catch API errors at runtime and diagnose the difference between the expected and actual schema.
- Hot-patch the agent's tool definitions in memory and log every patch.
- Complete the task after the patch, with no manual restart.
- Handle synthetic breaking mutations: field renames, type conversions, nesting changes and required headers.
- Measure adaptation rate, recovery time and token overhead against a baseline agent.

Core Challenge

The hard part is safe adaptation. The system must infer a correct new schema from error messages and responses, change only what the diff requires, and avoid silently passing hallucinated values that corrupt data. Each patch must be logged so it can be audited, and the agent must stop retrying once a patch fails, rather than looping.

Expected Deliverables

Must have: An agent that runs tasks against a mock API; a self-healing layer that catches errors, diagnoses the schema diff, hot-patches tool definitions and logs each patch; a live demo in which the mock API's schema changes on stage (for example, from {"user_id": "123"} to {"target": {"id": 123}} plus a new mandatory header); and a baseline agent built on a standard framework for comparison.

Metrics: Zero-Touch Adaptation Rate (share of synthetic breaking mutations resolved autonomously), MTTR (time from API break to recovery, ideally under 4 seconds), and Token Overhead (cost of diagnosing the diff compared with a failed, looped session).

Expected: A 30-second pitch, a short write-up of the diagnosis and patch logic, and a comparison showing the baseline crashing with a 422 and looping, while the self-healing agent completes the task in 2 to 3 seconds.